Internals

Architecture

```
Application (iced daemon, one update loop)
├── PresentationState        authoritative domain state         pulpit-core
├── DisplayCoordinator       snapshots, roles, reconcile()      pulpit-display
├── DocumentManager          watch, debounce, atomic reload     pulpit_app::doc
├── RendererSupervisor       worker pool, IPC, generations      pulpit-render
├── FrameCache               byte-bounded CPU/GPU accounting    pulpit-render
├── InputRouter              keymap incl. raw scancodes         pulpit_app::settings
├── SessionInhibitor         acquire/release, crash-safe        pulpit-app
└── Settings & Diagnostics   atomic, versioned, reportable      pulpit_app::settings
```

Four packages, not nine: `core`, `display` and `render` are separate because they cross a process or tool boundary, isolate a large external dependency, or have a test surface worth running alone. The rest were app-only libraries whose Cargo boundary bought nothing, and are now modules with the same rule — no Iced, no clocks, no services below the application layer.

Why the domain crates are pure

`pulpit-core`, the decision half of `pulpit-display`, and `pulpit_app::doc`'s state machine contain no UI types, no window handles, no PDF library types and no clock reads (time is passed in). That is what makes the hard cases — reconnect at a new index, an unequal mirror, a partial write, a stale delayed notification — ordinary unit tests that run in CI without a graphical session.

The three rules

1. One reconciliation function. `reconcile(snapshot, roles, capabilities, windows) -> Outcome` is pure and idempotent: applying its actions and calling it again produces nothing. Swap is a role exchange followed by ordinary reconciliation, never a sequence of ad-hoc window moves. Repeated notifications are free (`Reconciliation::Unchanged`) and older snapshots are dropped (`Reconciliation::Stale`).

2. No native handle survives an event-loop turn. `DisplayRoles` stores versioned identity *records*. Reconciliation enumerates afresh, resolves an identity to a live handle immediately before a native call, and forgets it. A monitor disappearing between resolution and use returns `PlacementOutcome::Disappeared`, which triggers another snapshot — not an error path.

3. The audience frame is never worse than it was.

- The audience window is created only when Start is pressed, initially hidden, assigned, and shown only with a valid frame (`Warning::AwaitingFirstFrame`). Stop destroys it.
- New pages are rendered coarse-first and refined afterwards, and the cache falls back across generations so a reload cannot blank the output. “Never worse” covers quality, not only absence: each output remembers what it is showing and swaps textures only for a meaningful improvement — the audience window holds its last frame rather than dip through a coarse one while the refined render is in flight, and a presenter panel walks at most one step up from its stand-in instead of blinking through the whole coarse-to-refined ladder. The presenter’s current-slide panel is a mirror of the projector — the same frame, chosen by the same rules, changing in the same beat the room sees — though blanking is not mirrored: the room’s screen goes dark, the presenter’s place in the deck does not. Every panel swap is logged at debug level.
- The render queue serves the committed audience page first, then the presenter panels’ preview-size frames, and only then the audience-size prefetch of the neighbouring pages: prefetch is a background luxury, and one such frame costs roughly ten panel frames. Warming covers two

pages behind the presenter as well as two ahead, so stepping backward is as seamless as stepping forward.
- A rebuilt PDF is promoted only after its first audience frame renders; a failed or partial build leaves the working document untouched and retries with bounded backoff.
- Renderer workers are separate processes. A crash fails one request, is reported, and is followed by a bounded restart.

Message flow

Everything — key presses, timer ticks, file-watch hints, topology hints, renderer replies — becomes an application message handled by one `update` function. Subscriptions are stable across view rebuilds, so no watcher or timer is ever duplicated. The renderer is pumped from the tick handler, which keeps IPC results inside the same single-threaded state transition as user input.

Cache accounting

Eviction is bounded by decoded bytes, never page count: a 3840×2160 RGBA frame is 33,177,600 bytes. CPU bitmaps and GPU textures are counted separately, the frames currently on screen — and the prefetched neighbours whose whole purpose is to survive until the next page turn — are pinned and never evicted, a frame larger than the whole budget is refused rather than allowed to evict everything, and the statistics are visible in diagnostics. A render request is satisfied only by a cached frame near its own width, so an audience-resolution frame never suppresses the panel-size render the presenter windows depend on. Deck thumbnails live on a separate budget and are rendered in one pass at one width — chosen per document so the whole deck fits — and are then immutable for the document's life, so no texture downstream ever swaps because of them.

Standing invariants

These are enforced today by the shipped runtime and must not be weakened by any later change. The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY** are normative.

The audience frame

1. **The audience window is output, not chrome.** It MUST contain only the presentation frame or an intentional black/white blanking frame — never notifications, controls, focus rings, diagnostics, dialogs, loading flashes, or runtime-selection UI. Blanking colours are exact output colours, independent of the application palette.
2. **A presenter-side animation, theme change, dialog, resize, or error MUST NOT invalidate the last valid audience frame.** After a session has produced a frame, its last complete frame is retained until a complete replacement arrives; partial buffers and worker errors never reach the audience. The `last_audience` cache key is pinned for this reason.
3. **The PDF page remains the fallback.** Without a media runtime, every overlay shows its poster or the PDF page and the deck still presents.
4. **Failure is presenter-side.** No known platform-specific failure may blank, replace, or expose controls on the audience frame.

Overlay runtime

1. **One session, two consumers.** Each overlay has exactly one authoritative session producing one ordered frame stream. The audience and presenter views consume the same frames and MUST NOT create separate decoders, browser tabs or JavaScript contexts.
2. **Capabilities, not operating-system branches.** A runtime is chosen by what it probes as able to do. A runtime lacking a required capability is skipped even when it claims the content type. A security-policy denial is never bypassed by trying a less restrictive runtime.

3. **Heavy runtimes stay out of process.** A worker crash MUST fail only its sessions. The main executable MUST NOT link a media engine, and packaging MUST verify this from the final executable's dynamic dependencies.
4. **One browser process per worker, one page per session.** A new session MUST take a page from the pool, never a process. Because the CDP pipe is shared, every message MUST be routed by the session id of the page it names — a session reading the pipe directly consumes another session's frames. A browser crash therefore fails every session in that worker together; each falls back independently.
5. **Frames are asked for at the overlay's size.** The screencast carries `maxWidth`/`maxHeight` so Chrome encodes the size Pulpit wants rather than the worker resampling. A viewport change MUST restart the screencast.
6. **A wrapper page draws no controls.** Anything the generated page paints is painted on the projector — a hover-revealed scrub bar is a scrub bar the room sees. Playback controls MUST live on the presenter's layout and reach the content through `MediaRequest::{Video, Image}`, which the worker translates into calls on the page's `window.__tp`. The page reports its playhead back through `__tpReport`. Host commands are built from a fixed vocabulary in the worker; there is deliberately no generic `eval`.

One runtime implements animated images, video and interactive HTML: `external-chromium`, an installed Chromium-family browser driven over the Chrome DevTools Protocol, never linked into Pulpit. The cost is recorded honestly: a machine with no such browser installed shows the fallback for *every* media overlay, not just for HTML. That is accepted, not a gap.

Platform boundary

1. **Ask what the session can do, never what OS it is.** Views and domain logic ask for capabilities — targeted fullscreen, arbitrary placement, system appearance, sleep inhibition, native menus, accessibility bridge, media keys. An unavailable capability produces one of three deliberate outcomes: a documented safe fallback, a specific manual action for the user, or a disabled command with an explanation. Silent no-ops and optimistic success messages are forbidden. Failure to target a chosen display MUST NOT be reported as success merely because some form of fullscreen was entered.
2. **Four contracts, one snapshot.** `DisplayBackend`, `PlatformServices`, `WindowPolicy` and `InputPolicy`, plus an immutable `Capabilities` snapshot refreshed when the session or display server changes. Adapters return data and explicit outcomes; they MUST NOT mutate presentation state.
3. **Pure crates stay pure.** No UI, OS, PDF-library or clock types in the domain, layout, document, rendering or settings models. Platform-only crates appear under target-specific `Cargo.toml` sections; conditional compilation belongs in adapter modules, not in state transitions or views. `unsafe` and FFI are confined to the smallest adapter module, each block documenting its lifetime, thread and ownership invariants.
4. **Persisted values are portable.** Settings, cache and log locations use platform-standard directories; paths are `Path`/`PathBuf`, not assumed UTF-8. Nothing persisted may encode a monitor index, absolute window position, OS font name, physical DPI, or platform shortcut spelling when a semantic representation exists. Migrations are deterministic and tested from fixtures of every supported platform.
5. **Native shell, Pulpit interior.** Ordinary windows use OS decorations; Pulpit MUST NOT draw a custom title bar to look the same everywhere. Inside the frame it uses one deliberate visual language rather than imitating GTK, WinUI or AppKit.

Visual and interaction system

1. **Seven colour roles, no other vocabulary.** `canvas`, `surface`, `slide_canvas`, `text`, `muted`, `accent`, `alert`. Borders, overlays, disabled and interaction states, and readable text on colour are derived centrally from those roles. Components MUST NOT invent aliases such as `primary`, `danger` or named hues. Spacing (4/8/12/16/24/32), radii and typography scales likewise live in the theme layer, not in views.
2. **Status is never conveyed by colour alone**, and contrast is at least 4.5:1 for normal text, 3:1 for large text, control borders, focus rings and meaningful graphics. High-contrast system modes take precedence over the selected palette.
3. **Logical units everywhere but rasterisation.** Layout, spacing, hit targets and persisted window sizes are logical pixels; slide rasterisation uses the destination's physical size and scale factor. A scale change triggers relayout and a resized render without blanking the audience. No view may assume 96 DPI, integer scale, or equal scale between displays. Pointer targets are at least 32 logical pixels, live controls at least 40.
4. **Keyboard first, pointer fully.** Every presenter and layout-editor operation is reachable by keyboard; focus order follows reading order and focus stays visible. Hover MUST NOT be the only way to reveal information or an action. Drag interactions have keyboard alternatives.
5. **Semantic commands.** Actions have stable identifiers independent of shortcut, menu position, label or platform. Keyboard, menus, buttons, remotes and automation dispatch the same action model.
6. **Motion communicates state, never decorates.** It MUST NOT delay navigation, audience updates or blanking, SHOULD complete within 200 ms, and is reduced when the system asks for reduced motion.
7. **Errors say what failed, what is currently safe, and what to do next.** A transient toast is never the sole record of a presentation-critical failure, and never appears on the audience window.
8. **Layouts are device-independent.** Normalised split proportions, minimum sizes in logical units, no monitor index, desktop coordinate, OS font name or physical DPI. The designer canvas MUST use the same layout algorithm as the live presenter view — a separate approximation is forbidden.

Performance

1. **Cost follows the active set, not history.** Off-page media sessions are parked (`Page.stopScreencast`, so they cost no browser encoding, no CDP transport and no worker decode) and evicted beyond a bounded parked count and parked ring-byte budget. The worker's poll loop pumps only sessions that are active or have queued events.
2. **Frames are delivered at the consumption rate, and only the newest.** Capture is capped (default 30 fps) by `everyNthFrame` plus an authoritative publish deadline; within one supervisor drain only each session's newest frame is copied. Superseded frames release their ring slot uncopied.
3. **No copy that ownership can avoid.** Exact-size frames go from decode straight to the ring; scratch buffers are reused rather than reallocated per frame; image handles share the frame cache's allocation (`Bytes::from_owner`) instead of deep-copying it; the annotation model is snapshotted behind an `Arc` with a revision counter and drawn through `canvas::Cache`, so unchanged strokes are neither cloned nor re-tessellated.
4. **Budgets must be honest.** A byte budget names what it actually counts ("source bytes"). Pinned overcommit is reported rather than hidden, and no permanently-zero figure is displayed. Full GPU/texture accounting remains unavailable through Iced and is stated as such rather than estimated silently.

5. **Measure before restructuring.** Two recorded negative results: replacing the CDP pipe's 1 ms retry sleep with `poll(2)` cost 2-3x the worker CPU for one to two milliseconds of latency (the sleep stays, with a comment saying why); and the 50 ms application tick MUST NOT be replaced until a wrapped GUI build has actually been profiled. Static-analysis findings are hypotheses until measurement attaches numbers to them, and debug builds never set targets.

Definition of supported

A desktop platform is **supported** only when the workspace builds from a clean checkout with documented prerequisites; the application is packaged with an icon, identity, required native libraries and platform-standard install/uninstall behaviour; presenter and audience windows pass the topology, mixed-DPI, fullscreen, reconnect and sleep/resume scenarios the platform permits; unsupported placement or fullscreen is detected and explained; native decorations, paths, shortcuts, dialogs and lifecycle rules are used; keyboard and screen-reader acceptance checks pass; no known platform failure can disturb the audience frame; and diagnostics identify the OS, session/window backend, renderer, display capabilities, scale factors and every fallback in effect. Physical display qualification is a release gate for each new platform adapter. Passing compilation alone is **experimental**, not supported.

A new runtime or adapter MUST plug into the same reconciliation and presentation state machines; it MUST NOT fork the application workflow.

Review checklist

- Does this preserve the last valid audience frame?
- Is this application behaviour or a platform service?
- Is the view asking for a capability rather than an OS name?
- Does it work with keyboard and assistive technology?
- Are focus, disabled, failure and fallback states visible and explained?
- Does it work at fractional scale and 200% text scaling?
- Is any saved value tied to a monitor index, pixel density, platform path or shortcut spelling?
- Does it use semantic tokens and commands?
- Is platform code confined to an adapter, tested with a null implementation?
- Would a failure affect only the presenter window?

The platform boundary

`pulpit_app::platform` is the only module that knows about D-Bus, portals, `xdg-open`, `%APPDATA%` or `~/Library`. Everything else talks to four traits plus a snapshot:

Contract	What it owns
`PlatformServices`	appearance, reveal/open, notifications, sleep inhibition, directories, recent documents
`WindowPolicy`	application id, minimum window size, quit-on-last-close, clamping restored bounds back onto a live work area
`InputPolicy`	the primary modifier, how a shortcut is written, which combinations the desktop has already reserved
`Capabilities`	a snapshot of what this session can actually do

`Platform::detect()` assembles them; `Platform::null()` gives a recording adapter that tests assert against without touching a desktop.

Outcomes, not booleans

Every operation that leaves the process returns `Outcome`:

```
Outcome::Done                          // it happened
Outcome::Refused { by, reason }        // the desktop said no, and why
Outcome::Unsupported { what }          // this session cannot do it at all
Outcome::Failed { reason }             // it should have worked and did not
```

The distinction is not pedantry. “Wayland will not let an application place its own windows” is a fact about the session that the presenter should be told once and calmly; “the compositor refused this specific request” is a fact about this attempt, worth retrying; “the call errored” is a bug. Collapsing all three into `false` is how an application ends up silently doing nothing on a projector.

Capabilities over OS checks

`Capabilities` reports the backend, the quality of display identity (`Stable` ⟶ `Connector` ⟶ `Geometric` ⟶ `None`), whether targeted fullscreen, arbitrary placement and safe un-fullscreening are possible, whether appearance and high contrast can be read, whether sleep can be inhibited, and whether native dialogs, menus, an accessibility bridge, media keys and notifications exist. `report()` renders it for the diagnostics bundle and the settings page; `limitations()` yields the ones worth telling the presenter about.

The X11 adapter claims placement; the portable Wayland adapter does not. On a Niri session, a runtime wrapper claims targeted placement through Niri’s IPC and moves each role-specific window to the selected output’s active workspace. The UI adapts on the resulting capability claim alone — never on `cfg!(target_os = ...)`.

The design system

`crates/pulpit-app/src/theme/tokens.rs` defines the seven colour roles shared by Settings and code. Borders, interaction states, and readable text over fills are derived from them. The same layer defines the 4/8/12/16/24/32 spacing scale, radii, a type scale, and hit-target sizes. Dark and light palettes are editable; system high contrast wins.

Contrast is a test, not a hope: the defaults keep body and muted text at 4.5:1 and meaningful graphics at 3:1. Settings warns when a custom pairing falls below those ratios, while foregrounds over fills are selected automatically.

Views read the palette through `theme::ambient`, which is set once per view pass. The explicit, palette-taking style builders underneath are what the tests exercise; the ambient layer only spares every widget from threading the palette by hand.

Status: toasts, and why they are never the whole story

Notices appear in the corner of the **presenter** window and never on the audience window. Routine information fades after four seconds, warnings after eight, and failures do not expire at all — a presentation-critical problem stays until it is dismissed, and carries a line saying what to do next. Repeating a message refreshes it rather than stacking a duplicate, and routine chatter can never evict a sticky failure.

Everything shown as a toast is also written to the diagnostics bundle. A toast is a courtesy; the bundle is the record.

Platform support today

Platform	Enumeration and identity	Targeted fullscreen	Notes
X11	XRandR + EDID	yes, via EWMH	reference platform
Wayland	`wl_output` + `xdg_output`	**no** — compositor placement, explained in the UI	needs the toplevel object, which Iced does not expose
Windows / macOS	none	falls back	not in this build

Iced 0.14 exposes no monitor enumeration and no targeted fullscreen; `crates/pulpit-display` implements both behind a trait, so an upstream contribution or a pinned patch can replace an adapter without touching the application.

Display-control findings

The original spike question was not "can we draw two windows" — it was **what does the platform actually permit, and what does parity with GTK cost?** These are the answers this codebase is built on, derived from the pinned versions in `Cargo.lock` (iced 0.14.0, winit 0.30.13) and from running the application on X11.

Iced's public API has no monitor story

`iced::window` (0.14) offers `set_mode(Windowed | Fullscreen | Hidden)`, `move_to`, `position`, `scale_factor` and `monitor_size(window)` — the *size* of the monitor containing a window, nothing more. There is no monitor enumeration, no monitor identity of any kind, no way to fullscreen onto a chosen output, and no monitor-connected or monitor-disconnected event.

`window::run(id, f)` hands a callback `&dyn HasWindowHandle + HasDisplayHandle` — raw window handles, not winit's `Window` — so even a targeted-fullscreen shim cannot be written against the public API alone. Two ways forward, in order of preference: contribute `Monitor` enumeration and `set_fullscreen(Some(monitor))` upstream, or carry a pinned `iced_winit` patch.

Monitor identity: what each tier costs on Linux/X11

Tier	Source	Available	Cost
1 — stable	EDID serial via XRandR `EDID` output property	yes, when the driver exports it	80 lines of EDID parsing, no new dependency
2 — connector + make/model	XRandR output name plus EDID descriptors	yes	free once tier 1 is parsed
3 — make/model/size/position	as above plus CRTC geometry	yes	free
4 — session handle	XRandR output id	yes	free, never persisted

On the reference machine the adapter produced a tier-1 identity from EDID for the built-in panel, so persisted display choices survive re-enumeration. The Wayland adapter (`smithay-client-toolkit`) reaches tier 2: connector name plus make and model.

Topology events

XRandR provides `RRScreenChangeNotify` / `RRCrtcChangeNotify` / `RROutputChangeNotify`; the adapter selects them and exposes `wait_for_change()`. They are treated strictly as **hints**: every reconciliation re-enumerates, and the snapshot carries a monotonic sequence number so a delayed notification from an older topology is dropped. The baseline remains a 1 Hz poll, which is what runs when no native listener exists.

Targeted fullscreen

On X11 with an EWMH window manager the sequence that works is: leave fullscreen, `ConfigureWindow` to the target monitor's origin, then `_NET_WM_STATE_FULLSCREEN`. Every placement request returns an explicit outcome — `Applied`, `Refused`, `Disappeared`, `Unsupported`, `Failed` — and the UI surfaces the ones the user must act on. **Refusal is detected by the absence of `Applied`, never assumed to be success.**

On Wayland `xdg_toplevel.set_fullscreen(output)` is the only mechanism, and it must be issued on the toolkit's own toplevel object, which Iced 0.14 does not expose. The adapter therefore reports `targeted_fullscreen: false`, and the application falls back to compositor fullscreen while saying so in the UI. This is the single capability that the upstream Iced change would unlock; nothing else in the design depends on it. A second, subtler limitation: this adapter opens its own Wayland connection, so its outputs are correlated with winit's windows by connector name rather than object identity.

Scale factor

X11 has no per-monitor scale. The adapter reports `Xft.dpi / 96` as a global hint, and the diagnostics bundle prints logical × scale = physical per monitor so a mismatch is visible rather than silently wrong. On the reference machine winit guessed 1.604 for a 2880×1800 panel while the adapter reported the Xft value; that real topology is committed as `tests/topology/08-captured-laptop-fractional-scale.txt`.

On Wayland the check is implemented: `WaylandBackend::scale_checks()` compares reported logical size × scale factor against the current mode's physical pixels for every output, and the result is logged at startup and included in diagnostics. What remains is *running* it on GNOME, KDE and a wlroots compositor.

Capability envelope

Encoded as `Capabilities { targeted_fullscreen, arbitrary_position, unfullscreen_safe, place_before_map }` and consumed by the single reconciliation function:

- **X11/EWMH**: everything true except `place_before_map`.
- **Wayland**: nothing placeable from here; unfullscreening is *not* safe, so the reconciler leaves a fullscreen audience window alone and says why (`CannotLeaveFullscreen`).
- **Niri/Wayland**: output enumeration still comes from Wayland, while Niri IPC identifies the role-specific window and sends it to the selected output's active workspace. Placement is retried after a hidden window is mapped.
- **Tiling WMs (i3/Sway)**: nothing is placeable; the reconciler emits `PlacementUnsupported`, keeps both windows visible and tells the user what to do. This is a supported configuration, not an unsupported one.

Pre-map placement, suspend and resume

Some window managers ignore placement issued before a window is mapped. Pulpit needs no configuration flag for it: a placement that is not `Applied` is queued and retried after the window exists, up to four times with increasing delay, after which the user is told what to do manually.

Linux's monotonic clock stops while the machine is asleep, so a tick gap far larger than the 50 ms interval — or a wall-clock gap that outruns the monotonic one — is a resume. On resume the application re-enumerates the topology, reconciles, and re-requests the frames both windows need, while page, preview, timer, blanking state and display roles are deliberately untouched.